Evaluating CNN Architectures and Data Augmentation for Object Recognition

Elia Stamm
Otto-Friedrich University of Bamberg
96049 Bamberg, Germany
elia-maximilian.stamm@stud.uni-bamberg.de

Project: xAI-Proj-B
Degree: B.Sc. AI
Matriculation #: 2104632

Dominic Schlegel
Otto-Friedrich University of Bamberg
96049 Bamberg, Germany
dominic-christopher.schlegel@stud.uni-bamberg.de

Project: xAI-Proj-B
Degree: B.Sc. AI
Matriculation #: 1984926

Magdalena Klug
Otto-Friedrich University of Bamberg
96049 Bamberg, Germany
magdalena_klug@stud.uni-bamberg.de

Project: xAI-Proj-B
Degree: B.Sc. AI
Matriculation #: 2154636

Abstract

This project investigates the impact of convolutional neural network architectures and data augmentation on image classification performance. Three different models were evaluated: a simple custom CNN (SimpleNet), an extended CNN with regularization techniques (LargerNet), and a pretrained ResNet18 model. All models were trained on a subset of ImageNet containing approximately 13,000 images across ten classes of everyday objects.

To evaluate model generalization, performance was measured on validation data. All models were tested on a separately collected outdoor test dataset consisting of 4409 images captured by student groups. Model performance was assessed using accuracy, per-class accuracy, confusion matrices, and ROC-AUC scores.

The results show that ResNet18 clearly outperformed both custom architectures on validation and test data. While data augmentation had limited impact on validation performance, it reduced performance on the outdoor test dataset for all models.

These findings suggest that pretrained feature representations are more robust under domain shift compared to models trained from scratch on limited data.

1 Introduction (E)

1.1 Project Format

The project was designed as a hands-on exploration of deep learning for image classification. We trained convolutional neural networks on a subset of ImageNet containing approximately 13,000 labeled images, focusing on ten everyday object classes: *coffee_mug, notebook, remote_control, soup_bowl, teapot, wooden_spoon, computer_keyboard, mouse, binder,* and *toilet_tissue*. Throughout the project, we systematically experimented with different network architectures, training strategies, and optimization techniques. Model performance was assessed using standard evaluation metrics, and we further examined the effect of data augmentation methods on classification accuracy. The project concluded with the presentation and analysis of experimental results, emphasizing both performance and methodological insights.

Beyond standard benchmarking, the project included an additional evaluation step aimed at testing model robustness. After training, all architectures were evaluated on a separately collected test set containing images of the same classes but captured in outdoor environments. Each student contributed at least 300 annotated images, and the final test set combined the data from all participants. This setup allowed us to assess how well the trained models generalized to a shifted data distribution, rather than merely measuring performance on data similar to the training set.

1.2 Motivation and Goals

Our primary goal was to compare different convolutional neural network architectures in the context of an image classification task with limited training data. This constraint made overfitting a central challenge and motivated us to explore architectural and regularization choices that could improve generalization. By progressing from simpler models to more advanced architectures, we aimed to understand how design decisions affect performance and robustness, rather than solely optimizing for accuracy.

In addition, we placed strong emphasis on systematic model evaluation. Precise evaluation metrics were essential for comparing architectures fairly and for assessing the impact of different data augmentation strategies. By evaluating models not only on standard validation data but also on a separately collected test set, we sought to better understand how training choices influenced generalization to unseen and more challenging data distributions.

2 Methodology

2.1 Data Augmentation (D)

To increase the diversity of the training data, we applied a limited set of commonly used image data augmentation techniques during training. Data augmentation is a standard approach in image classification to reduce overfitting and to improve generalization by exposing models to variations of the input data (Shorten and Khoshgoftaar, 2019).

The applied augmentations include basic geometric and photometric transformations. Geometric augmentations were used to introduce robustness to changes in object position and scale, which frequently occur in real-world images. In our setup, input images were first resized to 272×272 pixels and then randomly cropped to 256×256 pixels. The random crop was sampled using a scale range of $(0.85, 1.0)$ and an aspect ratio range of $(0.9, 1.1)$. In addition, horizontal flipping was applied with a probability of 0.5, which is commonly used in object recognition tasks (Shorten and Khoshgoftaar, 2019).

To account for variations in illumination and color conditions, we further applied photometric augmentations in the form of brightness, contrast, and saturation adjustments. Such color-based transformations help models cope with changes in lighting and appearance in real-world image data (Shorten and Khoshgoftaar, 2019). In our configuration, brightness and contrast were adjusted by up to 0.12, while saturation was adjusted by up to 0.08.

All data augmentation techniques were applied exclusively during training. Validation and test images were processed without random augmentations.

2.2 SimpleNet Architecture (E)

The SimpleNet architecture was our initial CNN model, designed to be small and straightforward. Its simplicity allowed us to quickly implement and test basic convolutional and pooling operations, serving as a clear proof-of-concept for our image classification pipeline.

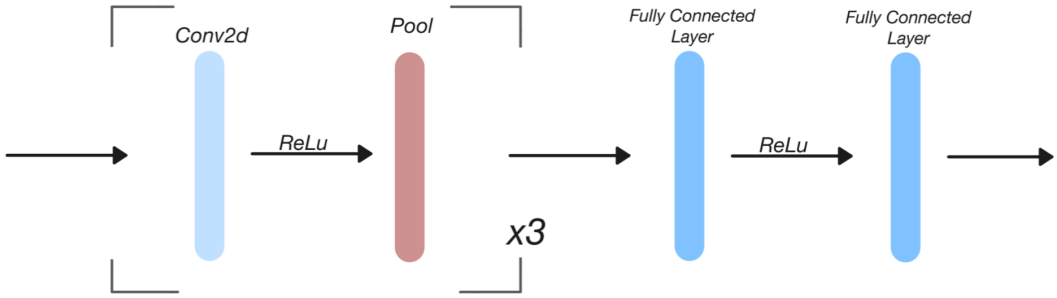

Figure 1: The SimpleNet architecture

The *SimpleNet* architecture consists of three repeated blocks of convolution, ReLU activation, and pooling layers, followed by a fully connected layer with ReLU activation, and a final fully connected layer for classification. This straightforward design allowed the network to extract hierarchical features from the input images while keeping the number of parameters low.

We used the Rectified Linear Unit (ReLU) as an activation function throughout SimpleNet due to its simplicity and effectiveness. ReLU introduces nonlinearity by applying a non-saturating activation, which enables the network to learn complex decision functions without altering the receptive fields of convolutional layers. Traore et al. (2018) show that ReLU significantly accelerates training compared to saturated nonlinear functions such as sigmoid or tanh, while maintaining comparable generalization performance. These properties made ReLU a suitable choice for our architectures.

Despite its modest size, SimpleNet provided valuable insights into the impact of architectural choices, such as layer depth, activation functions, and regularization techniques. Retaining it in the project highlights a baseline performance and allows for direct comparison with more advanced architectures, demonstrating how network complexity and design improvements influence accuracy and generalization.

2.3 LargerNet Architecture (E)

In the architecture we refer to as *LargerNet*, our goal was to explicitly address overfitting. We did this both by increasing the depth of the network and by incorporating several design choices motivated by our literature review and prior experimentation.

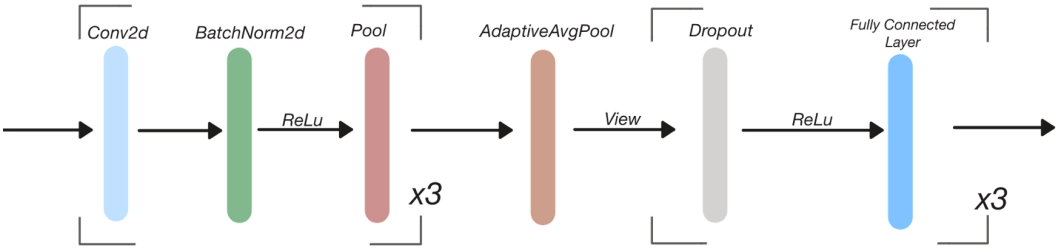

Figure 2: The LargerNet architecture

We added batch normalization to our CNN as part of an iterative effort to make training more stable and to improve generalization. Prior work by Ogundokun et al. (2022) shows that batch normalization accelerates convergence, reduces internal covariate shift, and acts as an effective regularizer in convolutional networks. These properties allow higher learning rates and help mitigate overfitting, especially in deeper architectures. Based on these results, we incorporated batch normalization layers

into our model to promote more robust feature learning and to reduce sensitivity to variations in the input data distribution, while maintaining efficient and stable training throughout our experiments.

Ogundokun et al. (2022) also show that combining dropout with batch normalization improves generalization by preventing co-adaptation of neurons during training. Motivated by these findings, we applied dropout in our network to encourage more robust feature learning.

The *LargerNet* architecture further differs from our earlier models by increasing overall capacity and structural depth. An Adaptive Average Pooling layer is placed between the convolutional and fully connected parts of the network to produce a fixed-size representation, allowing the model to better aggregate spatial information while remaining flexible to input variations.

2.4 ResNet18 Architecture (E)

We selected ResNet18 for its combination of practicality and strong performance. Pretrained weights are readily available in PyTorch, which allowed us to integrate the model efficiently into our setup. Its original training on ImageNet also aligned well with our task, providing feature representations that could be transferred effectively.

Moreover, ResNet18 is widely cited in the literature and is known for its residual architecture, which enables deep networks to learn complex features while avoiding vanishing gradient issues (Liang, 2020). These advantages made it a natural choice for our 10-class classification problem, offering a balance of reliability, efficiency, and strong baseline performance.

To adapt ResNet18 to our 10-class task, we replaced the original final fully connected layer, which outputs 1,000 classes, with a new layer matching our target classes. During training, we froze the weights of all other layers in the network, effectively using the pretrained backbone as a fixed feature extractor. Only the newly added classification layer was trained initially, allowing the network to quickly learn task-specific mappings without overfitting. After this stage, we optionally fine-tuned the entire model by unfreezing some or all layers, enabling the pretrained features to adjust slightly to our data while still leveraging the robustness of the original ResNet18 representations.

Residual blocks in ResNet18 allow input signals to bypass one or more layers through shortcut connections, enabling information to flow directly across the network. This design helps mitigate the vanishing gradient problem and allows very deep networks to be trained effectively. Even though our network was not extremely deep, the residual connections still supported stable training and improved feature propagation, making it easier for the model to learn complex representations without degradation as layers increased.

2.5 Training (M)

Model training was conducted using the Adam optimizer (Kingma and Ba, 2017) with default settings (initial learning rate of 0.0001). To ensure convergence and prevent overfitting, we employed a learning rate scheduler and an early stopping mechanism.

The learning rate scheduler implemented a ReduceLROnPlateau strategy, which reduces the learning rate by a factor of 0.5 when the validation loss fails to decrease over a patience period of 2 epochs. This adaptive learning rate adjustment enables finer gradient updates as the model approaches convergence.

Early stopping was applied to halt training when model performance on the validation set ceased to improve. Specifically, training was terminated if the validation loss did not decrease by a minimum delta of 0.1 over a patience period of 5 epochs. This approach prevents overfitting while minimizing computational overhead (Prechelt, 2002).

The training procedure followed an iterative process over multiple epochs: each epoch consisted of a training phase followed by validation. Subsequently, performance metrics were computed and logged, the learning rate scheduler was invoked to adjust the learning rate if necessary, model checkpoints were saved if the current validation metrics have been better than previous epochs and the early stopping criterion was evaluated to determine training continuation. If early stopping hasn't been triggered, the training stops after a total of 300 epochs.

2.6 Evaluation Metrics (M)

Model performance was assessed using multiple evaluation metrics to provide a comprehensive analysis of classification quality.

2.6.1 Accuracy

Accuracy was calculated as the percentage of correct predictions relative to the total number of predictions. For multi-class classification, accuracy is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

where TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively. In the context of this study, the confusion matrix extends to 10×10 dimensions, accommodating all class predictions.

2.6.2 ROC and AUC

Receiver Operating Characteristic (ROC) curve analysis and the corresponding Area Under the Curve (AUC) were employed to evaluate the model's ability to distinguish between classes (Hand and Till, 2001). For multi-class classification, we adopted a One-vs-Rest (OvR) approach following Fawcett (2006), calculating individual ROC curves for each class against all other classes. The True Positive Rate (TPR) and False Positive Rate (FPR) for each class are computed as follows:

$$\text{TPR} = \frac{TP}{TP + FN} \tag{2}$$

$$\text{FPR} = \frac{FP}{FP + TN} \tag{3}$$

The overall model performance was quantified using the weighted-averaged AUC, which computes the weighted mean of individual class AUC values, thereby treating all classes regarding their sample size. This approach provides a balanced assessment of model performance across all classes.

3 Experiments

3.1 Training Setup (E)

All experiments were conducted using the free-tier Kaggle environment, which provides up to 30 hours of GPU time per week on an NVIDIA P100. During initial experiments, we used a batch size of 64, which led to an imbalance between CPU and GPU utilization, characterized by prolonged periods of high CPU load and relatively low GPU usage. This indicated that data loading and preprocessing had become a bottleneck in the training pipeline. By reducing the batch size to 4, we achieved more consistent GPU utilization and significantly lower CPU overhead, resulting in a more efficient training process. Accordingly, we configured the PyTorch `DataLoader` with `batch_size=4`, `num_workers=4`, and a fixed random seed of 42 to ensure reproducibility and stable performance.

3.2 Architecture Comparison (M)

Table 1 summarizes the training results for each of the implemented neural network architectures.

SimpleCNN demonstrated the lowest performance across all metrics. The non-augmented variant stopped training early after only 8 epochs and showed a larger validation-training loss difference compared to the augmented version.

LargerNet achieved intermediate performance levels between SimpleCNN and ResNet. The augmented variant completed the full training cycle of 300 epochs with a smaller loss difference, while

the non-augmented version stopped after 91 epochs. Both variants achieved similar accuracy and AUC/ROC scores.

ResNet exhibited the highest performance among all tested architectures, with both variants completing 300 epochs of training. The augmented and non-augmented models achieved comparable accuracy and AUC/ROC values, with the augmented variant showing a smaller validation-training loss difference.

Across all architectures, data augmentation consistently reduced the gap between validation and training loss, with the most substantial effect observed in SimpleCNN.

Table 1: Training results on validation data for different neural network architectures with and without data augmentation.

Model	**SimpleCNN**		**LargerNet**		**ResNet**	
Augmentation	Yes	No	Yes	No	Yes	No
Epochs trained	30	8	300	91	300	300
Learning Rate	0.0001	0.00005	1.5×10^{-8}	1.5×10^{-8}	1.5×10^{-8}	1.5×10^{-8}
Loss Difference (Val. - Train.)	1.154	6.794	0.102	0.261	0.069	0.203
Accuracy	0.367	0.279	0.447	0.476	0.893	0.879
AUC / ROC	0.785	0.683	0.838	0.848	0.994	0.992

4 Results (D)

All results in this section are based solely on the self-collected outdoor test dataset. This dataset was not used during training or validation and differs from the ImageNet-based training data in terms of environment and image conditions.

The self-collected test dataset consists of 4409 outdoor images covering the same ten object classes as the training data. All images were captured by four independent student groups in real-world outdoor environments. The dataset includes variations in background, lighting, and viewpoints and was used exclusively to evaluate model performance under domain shift.

The evaluation aims to analyze how well the trained models generalize to unseen outdoor data. Performance is reported using overall accuracy, confusion matrices, per-class accuracy, and ROC-AUC scores.

4.1 Results on Testset (D)

Performance on the self-collected outdoor test dataset is compared across all evaluated models.

Table 2 reports the overall classification accuracy and ROC-AUC scores for SimpleNet, LargerNet and ResNet18. Each model is evaluated with and without data augmentation.

Across all evaluation settings, ResNet18 achieves the highest performance on the outdoor test set in terms of both accuracy and ROC-AUC. The two custom architectures, SimpleNet and LargerNet show lower performance on the same data. LargerNet slightly outperforms SimpleNet, while both remain clearly below the performance level of ResNet18.

The relative performance ordering of the evaluated architectures remains consistent across both evaluation metrics.

Table 2: Test results on the self-collected outdoor dataset for different neural network architectures with and without data augmentation.

Metric	**SimpleNet**		**LargerNet**		**ResNet18**	
Augmentation	Yes	No	Yes	No	Yes	No
Accuracy	0.124	0.141	0.140	0.155	0.326	0.467
ROC-AUC (OvR)	0.554	0.560	0.569	0.582	0.770	0.847

4.2 Effects of Data Augmentation (D)

The effect of data augmentation on test performance is evaluated by comparing augmented and non-augmented variants of all models. The corresponding accuracy and ROC-AUC values are reported in Table 2.

For all three architectures, data augmentation results in lower classification accuracy compared to the non-augmented variants. A similar decrease can be observed for the ROC-AUC scores. The largest absolute performance drop occurs for ResNet18, while SimpleNet and LargerNet show smaller differences between the two configurations.

4.3 Class Separability (D)

Class separability is analyzed using the best-performing model, ResNet18 without data augmentation.

Figure 3 shows the per-class classification accuracy on the outdoor test dataset. The figure indicates noticeable differences in classification performance across object categories. While classes such as wooden-spoon and toilet-tissue achieve comparatively high accuracy, other classes such as soup-bowl and teapot show lower accuracy values, suggesting that these classes are more difficult to distinguish in outdoor settings.

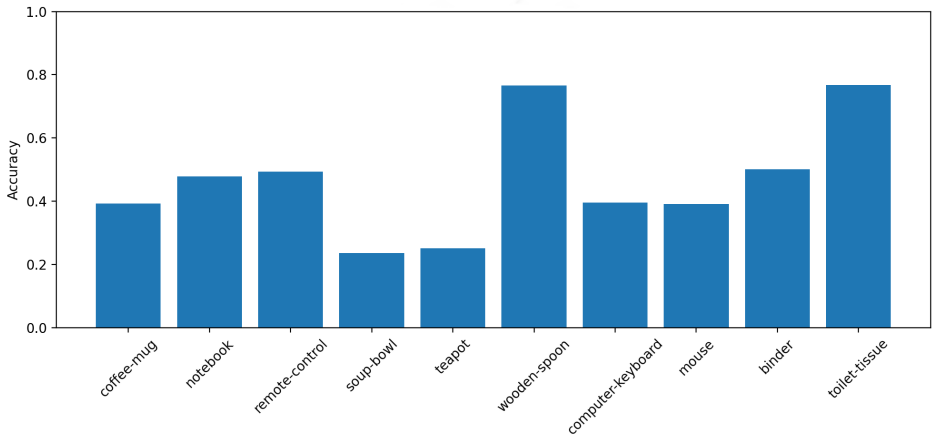

Figure 3: Per-class classification accuracy for ResNet18 without data augmentation on the test dataset.

To analyze the underlying error patterns in more detail, Figure 4 presents the corresponding confusion matrix. The matrix shows that most predictions are located on the diagonal, indicating correct classification for a large portion of the test samples. However, several off-diagonal entries reveal recurring confusions between specific class pairs.

In particular, images of teapots are frequently misclassified as toilet tissue, with 160 such cases visible in the confusion matrix. Similarly, coffee mugs are misclassified as toilet tissue in 121 instances, while binders are predicted as notebooks in 79 instances. Another prominent confusion occurs between computer keyboards and remote controls, where 77 keyboard images are classified as remote controls. These confusion patterns correspond to the lower per-class accuracy values observed for the affected classes in Figure 3.

5 Discussion

5.1 Discoveries (M)

The experimental results show distinct performance differences between the evaluated architectures and provide insight into the effects of domain shift on model generalization.

ResNet18 outperformed both custom architectures across all evaluation settings. On the validation data, ResNet18 achieved an accuracy of 0.893 (augmented) and 0.879 (non-augmented), compared

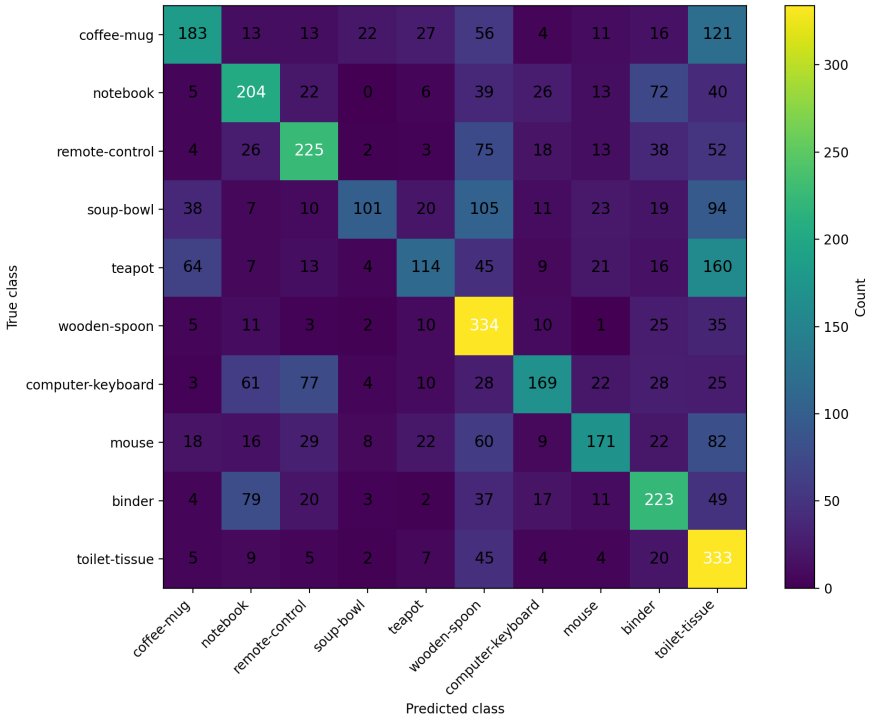

Figure 4: Confusion matrix for ResNet18 without data augmentation evaluated on the test dataset.

to LargerNet's 0.447–0.476 and SimpleNet's 0.279–0.367. This performance difference increased on the outdoor test set, where ResNet18 achieved accuracies of 0.326 and 0.467 while both custom architectures remained below 0.155. The ROC-AUC analysis shows a similar pattern, with ResNet18 achieving 0.847 (non-augmented) on the test set compared to 0.582 for LargerNet and 0.560 for SimpleNet. These results indicate that pretrained features from ImageNet transfer to the outdoor domain more effectively than features learned from scratch on the limited training data.

The performance of ResNet18 can be attributed to several architectural factors. The residual connections enable gradient flow during backpropagation, which facilitates the learning of deeper feature hierarchies (Liang, 2020). The pretrained weights provide initialization with features learned from a larger dataset. The model's higher parameter count allows for more complex decision boundaries. In contrast, SimpleNet and LargerNet were trained from scratch on approximately 13,000 images, which limited their ability to learn generalizable features.

Data augmentation reduced performance on the outdoor test set across all architectures while maintaining comparable performance during training. For ResNet18, augmentation reduced test accuracy from 0.467 to 0.326, a decrease of 0.141. Similar decreases were observed for SimpleNet and LargerNet. This pattern differs from the common observation that augmentation improves generalization.

This result may be explained by a mismatch between the augmentation strategies and the characteristics of the test domain. The augmentation techniques applied during training included random flipping, color jittering, and cropping. These transformations may not reflect the distribution shift between indoor ImageNet images and outdoor photographs. Outdoor images differ from ImageNet images in lighting conditions, background complexity, and perspective variations. The applied augmentations may have reduced the model's ability to learn features relevant to the outdoor test distribution.

The per-class analysis shows variation in classification performance across object categories in outdoor settings. Wooden Spoon and Toilet Tissue achieved higher accuracy than other classes. Classes such as Teapot, Mouse, Soup Bowl, and Computer Keyboard showed lower accuracy and higher confusion rates. The confusion matrix shows that teapots were misclassified as toilet tissue

in 160 instances, coffee mugs as toilet tissue in 121 instances, binders as notebooks in 79 instances, and computer keyboards as remote controls in 77 instances. These confusions occur between objects with similar shapes, which suggests that shape-based features may be insufficient when objects share geometric properties or when outdoor conditions alter visual appearance.

5.2 Limitations (M)

The study has several limitations. First, the custom architectures were trained from scratch on approximately 13,000 images. This limited the models' ability to learn generalizable features, particularly for SimpleNet, which stopped training after 8 epochs in the non-augmented setting. LargerNet employed batch normalization and dropout, but performance remained below ResNet18, which was pretrained on the full ImageNet dataset.

Second, the data augmentation strategy employed generic transformations without consideration of the target domain characteristics. As discussed above, this may have caused a mismatch between augmented training data and the test distribution. Following (Perez and Wang, 2017) augmentation typically increases the generalizability. However, this is not applicable in our case, as we have a domain shift between indoor and outdoor contexts. Domain-specific augmentation techniques that reflect outdoor environment variations, such as natural lighting conditions, background variations, and perspective changes, were not applied.

Third, the evaluation was limited to accuracy, per-class accuracy, and ROC-AUC metrics. These metrics quantify classification performance but do not indicate which visual features the models use or why specific confusions occur. Interpretability techniques such as Grad-CAM (Gradient-weighted Class Activation Mapping, Selvaraju et al. (2016)) were not applied. Such methods could identify which image regions contribute to predictions (Selvaraju et al., 2016) and explain the failure modes observed on the outdoor test set.

Finally, the outdoor test set contained images collected by students in the timespan between October and December. This may introduce biases in photographing style, object selection, and environmental conditions. A larger and more diverse test set would provide additional insight into model generalization.

6 Conclusion (D)

In this project, we compared different convolutional neural network architectures and evaluated the impact of data augmentation on image classification performance. The results show clear differences between pretrained and custom-trained models. ResNet18 consistently achieved the best performance on both validation and outdoor test data, while SimpleNet and LargerNet showed lower generalization capability, especially under domain shift.

The evaluation on the self-collected outdoor dataset highlights the importance of robust feature representations when models are applied to data outside the original training distribution. While data augmentation helped reduce overfitting during training, it consistently reduced performance on the outdoor test dataset. This suggests that generic augmentation strategies are not always sufficient when the target domain differs strongly from the training data.

Future work could focus on collecting more diverse training data and exploring augmentation strategies that better reflect real-world outdoor conditions. In addition, further analysis of model errors could help to better understand failure cases and improve model robustness.

References

T. Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. ISSN 0167-8655. doi: https://doi.org/10.1016/j.patrec.2005.10.010. URL https://www.sciencedirect.com/science/article/pii/S016786550500303X. ROC Analysis in Pattern Recognition.

D. J. Hand and R. J. Till. A simple generalisation of the area under the roc curve for multiple class classification problems. *Machine learning*, 45(2):171–186, 2001.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization, 2017. URL https://arxiv.org/abs/1412.6980.

J. Liang. Image classification based on resnet. *Journal of Physics: Conference Series*, 1634:012110, 2020. doi: 10.1088/1742-6596/1634/1/012110. URL https://doi.org/10.1088/1742-6596/1634/1/012110.

R. O. Ogundokun, R. Maskeliunas, S. Misra, and R. Damaševičius. *Improved CNN Based on Batch Normalization and Adam Optimizer*, pages 593–604. Springer International Publishing, 2022. doi: 10.1007/978-3-031-10548-7_43. URL https://doi.org/10.1007/978-3-031-10548-7_43.

L. Perez and J. Wang. The effectiveness of data augmentation in image classification using deep learning. *arXiv preprint arXiv:1712.04621*, 2017.

L. Prechelt. Early stopping-but when? In *Neural Networks: Tricks of the trade*, pages 55–69. Springer, 2002.

R. R. Selvaraju, A. Das, R. Vedantam, M. Cogswell, D. Parikh, and D. Batra. Grad-cam: Why did you say that? visual explanations from deep networks via gradient-based localization. *CoRR*, abs/1610.02391, 2016. URL http://arxiv.org/abs/1610.02391.

C. Shorten and T. M. Khoshgoftaar. A survey on image data augmentation for deep learning. 6(1):60, 2019. ISSN 2196-1115. doi: 10.1186/s40537-019-0197-0. URL https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0.

B. B. Traore, B. Kamsu-Foguem, and F. Tangara. Deep convolution neural network for image recognition. *Ecological Informatics*, 48:257–268, 2018. doi: 10.1016/j.ecoinf.2018.10.002. URL https://doi.org/10.1016/j.ecoinf.2018.10.002.

A Appendix

A.1 Code Availability

The implementation used for model training, evaluation, and result analysis is available at:

`https://github.com/elia09812/xai_proj_b_group_404`

Declaration of Authorship

Ich erkläre hiermit gemäß § 9 Abs. 12 APO, dass ich die vorstehende Seminararbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Des Weiteren erkläre ich, dass die digitale Fassung der gedruckten Ausfertigung der Seminararbeit ausnahmslos in Inhalt und Wortlaut entspricht und zur Kenntnis genommen wurde, dass diese digitale Fassung einer durch Software unterstützten, anonymisierten Prüfung auf Plagiate unterzogen werden kann.

Bamberg, February 8, 2026

(Ort, Datum)	(Unterschrift)

Bamberg, February 8, 2026

(Ort, Datum)	(Unterschrift)

Bamberg, February 8, 2026

(Ort, Datum)	(Unterschrift)